

Challenge 1:

The implementation of health bars inside the game.

Implementing health bars seemed easy enough as we covered a lesson on sliders before:

Purpose:

- Showcase HP between both players, this adds more agency in the game itself.
- Adds an added level of stress when the enemy hits your very hard

How will it function?

- Utilize the sliders with UI images to create a dynamic health bar.
- Must have the max hp/value be the hp of the unit.
- Use GetComponent to grab a hold of the HP float in the player/enemy script to use.
- Create a separate function that is called every time inside update.
-

Issues that arose during.

- The hp was not budging when hit,
 - o Found out that it was since I was required to put the casting function inside a separate user defined function.
 - o Calling the function inside update worked.

Big notes: The script being separated into 2 for each enemy and the player is a good option to keep things more organized. It is copy and paste anyways.

Challenge 2:

The implementation of the perfect dodge mechanic.

As discussed in class, use the animation events built into unity to trigger Booleans turning on and off. This will allow the simulation of IV (invincibility) frames inside my game.

Purpose:

- Add a skill ceiling to the gameplay.
- As of right now because of this mechanism I can boost the difficulty of the base game.
- Because of this I can instead of using flat damage numbers, I can use a damage formula with actual stats. Which can branch into an expansive relic system which I showcased a small bit of.

How will it function:

- Utilize animation events and 2 functions in each animation.
- Turn Iv frame Boolean true on first animation event, turn it off on second.
- Have another Boolean on the enemy side checking if it is attacking Aswell.
- Use a conditional statement checking if both attacks is true and if it is false, if yes then do damage, else negate.

Issues found during:

- If the frames are not perfectly aligned, then it is very hard to make it work properly.
- Originally, I only had a single boolean but after working through it I realized the IsAttacking boolean from the enemy also plays a gigantic role.

Challenge 3:

Make a damage system that can be easily modified but still retains the core of the combat mechanics.

The goal was to instead of using flat damage number like - 20 hp every hit I would use a mathematical formula and personal stats the characters have, to calculate damage instead.

- The formula in the end I used was the more standard approach of $\text{attack stat} * 2 - \text{enemy defense stat}$.
- This would result in damage.

Issues that happen with this approach:

- Without a plethora of attacks, stats are bland and can cause a fight to be just a stat check.
 - o To combat this the combat itself is a timing based difficult skill fight in which perfect dodging is a requirement to win.
 - o This allows for the user to equip different relics depending on their own playstyle.

Implementation:

- Both the player and the enemy have 3 float variables, for attack, defense, and hp. Relics will be called Booleans which will trigger a conditional to add stats to the player. This lets the code be very expandable and will not affect the stats of others if I ever wanted to swap.